UNIT – V: Ensemble Learning

Bias – Variance Trade-off – Bagging and Boosting (Random forests, Ada boost, XG boost inclusive) – Metrics & Error Correction.

Bias-Variance Trade-off

It explains the balance between two types of errors in machine learning models:

1. **Bias**: Error due to overly simplistic models (underfitting).
   - **Example**: A linear regression model used for non-linear data.
   - **Consequence**: Misses important patterns, leading to poor performance on both training and test data.
2. **Variance**: Error due to overly complex models (overfitting).
   - **Example**: A deep decision tree fitting every detail in the training data.
   - **Consequence**: Performs well on training data but poorly on test data, failing to generalize.

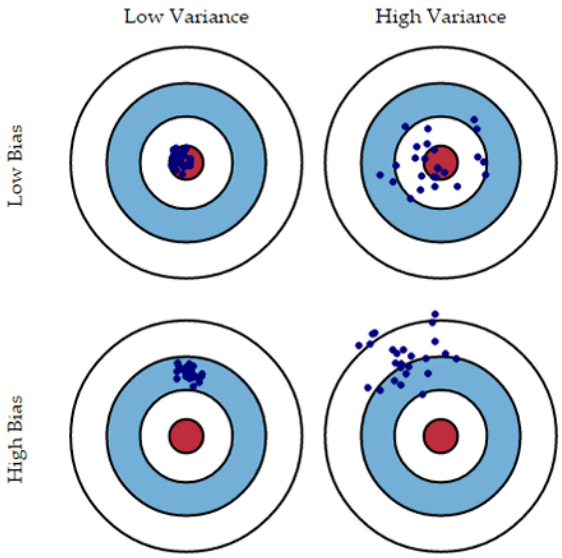
Consequences of Imbalance:

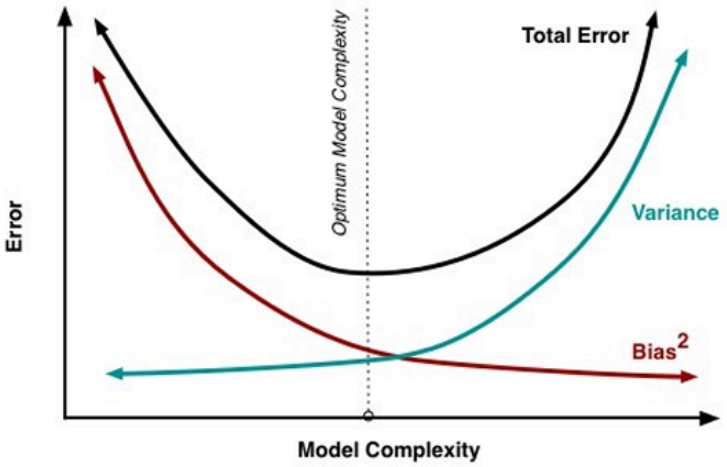
- **High Bias (Underfitting)**:
  - Low training and test accuracy.
  - Misses key patterns in the data.
  - Example: Using a simple linear model on a complex dataset.
- **High Variance (Overfitting)**:
  - High accuracy on training data but poor on test data.
  - Captures noise instead of general patterns.
  - Example: A very deep decision tree fitting every detail, including noise.

Goal: Minimize the total error by balancing bias and variance for optimal generalization to unseen data.

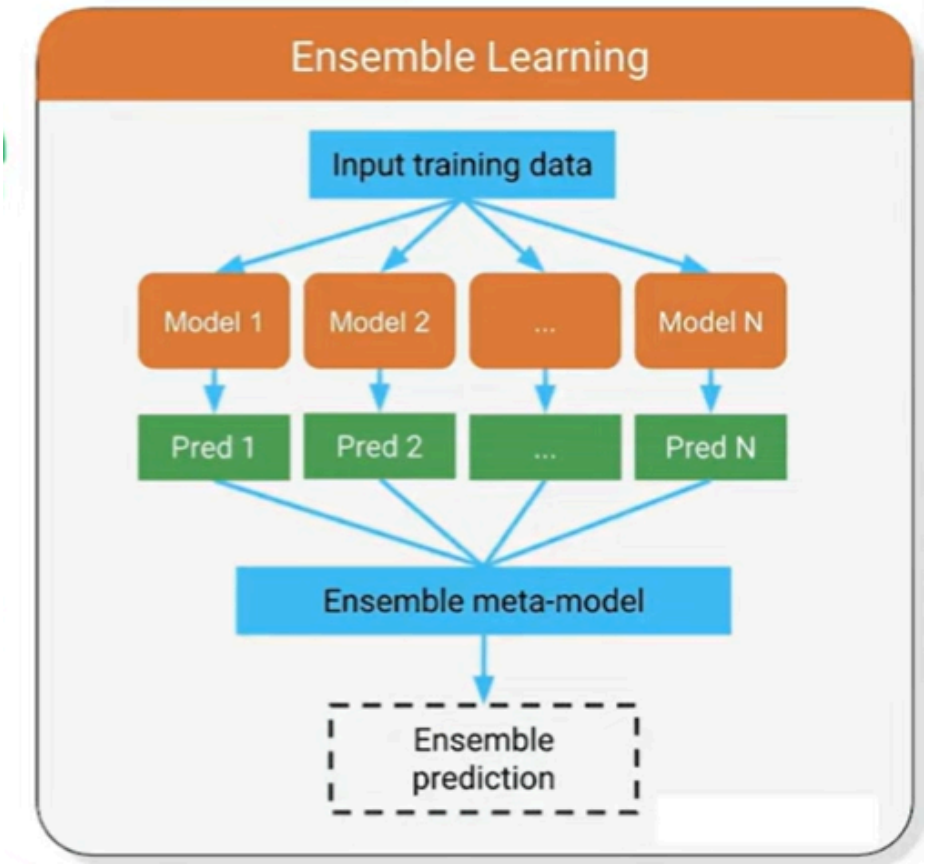
Key Strategies:

- **Regularization**: Techniques like Lasso or Ridge regression control model complexity, reducing variance.
- **Cross-validation**: Helps find the optimal model complexity to avoid underfitting or overfitting.
- **Ensemble Methods**:
  - **Bagging (e.g., Random Forest)**: Reduces variance by averaging multiple models.
  - **Boosting (e.g., AdaBoost)**: Reduces bias by focusing on difficult cases.





Ensemble learning is a machine learning technique where multiple models (often called "learners") are combined to solve a problem. The idea behind ensemble learning is that by combining the predictions of several models, the overall performance can be improved, often yielding better results than any single model on its own.



There are two main types of ensemble learning:

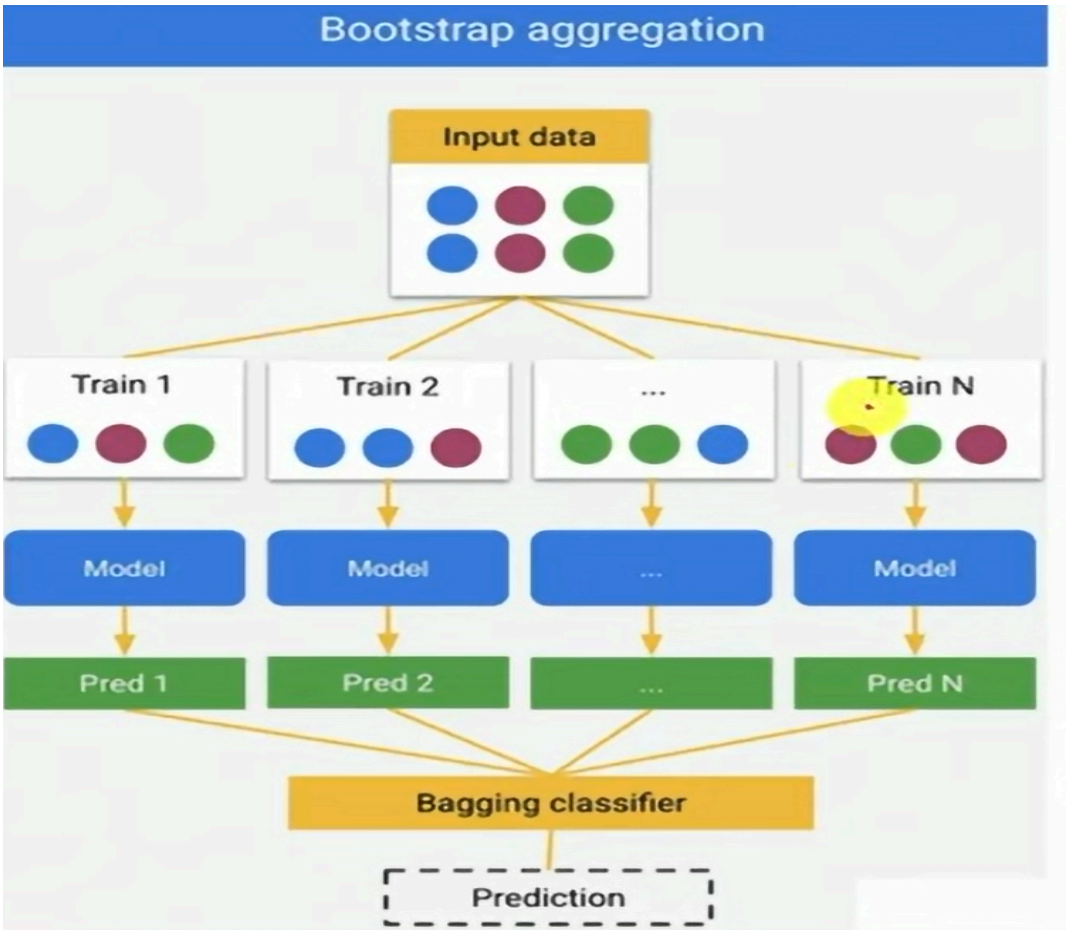
1. Bagging (Bootstrap Aggregating)

- Bagging creates multiple models by training each model on a different subset of the data, usually through bootstrapping (sampling the data with replacement).
- After training, the predictions of the individual models are combined (averaging for regression or voting for classification) to make the final prediction.

Examples:Random Forest: A collection of decision trees where each tree is trained on a different random subset of the data.

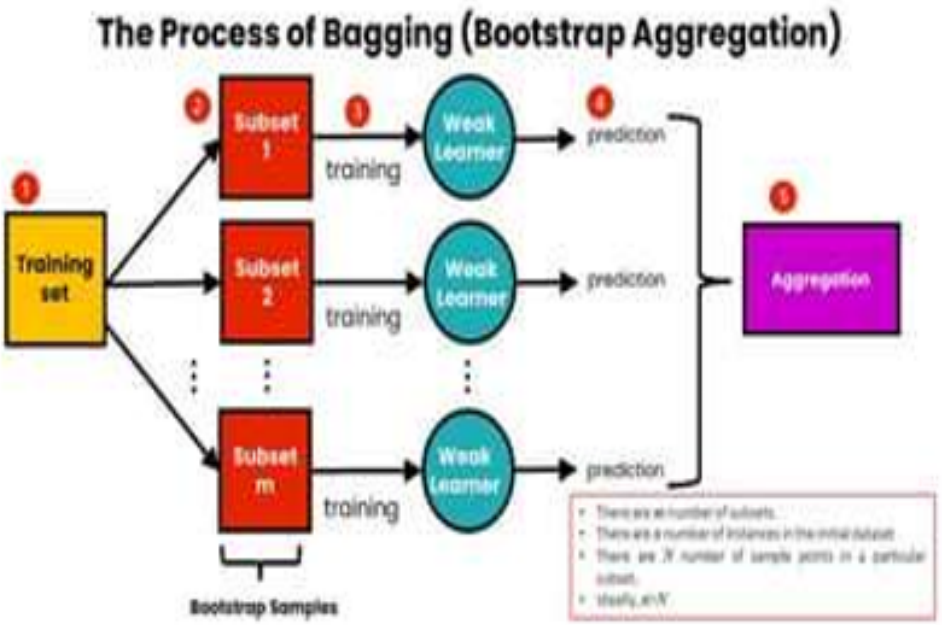
Advantages of Bagging:

- **Reduces Variance**: Since bagging uses different subsets of the data, it reduces the variance of the model by averaging the predictions. This helps in preventing overfitting, especially in models like decision trees that are prone to overfitting.
- **Improved Accuracy**.
- **Parallelizable**: The training of individual models is independent, meaning it can be parallelized easily across multiple processors, speeding up the computation.
- **Handles Noise Well**: Bagging can work well with noisy datasets, as averaging the predictions from many models reduces the impact of outliers and noise.



Disadvantages of Bagging:

- **Not Good for High-Bias Models**: If the base model has high bias (e.g., a simple linear regression model), bagging won't improve its performance significantly, as it mainly reduces variance, not bias.
- **Resource Intensive**: Since it trains multiple models independently, bagging can be computationally expensive, especially with large datasets.
- **Less Interpretability**: The combination of many models (e.g., in Random Forests) makes it harder to interpret the final model compared to a single decision tree or linear model.



2. Boosting

- Boosting involves training multiple models sequentially, where each new model corrects the errors made by the previous models.
- In boosting, models are trained by focusing on the errors (misclassifications) made by earlier models. These models are combined with weighted voting or summing, where more importance is given to the more accurate models.

Examples:

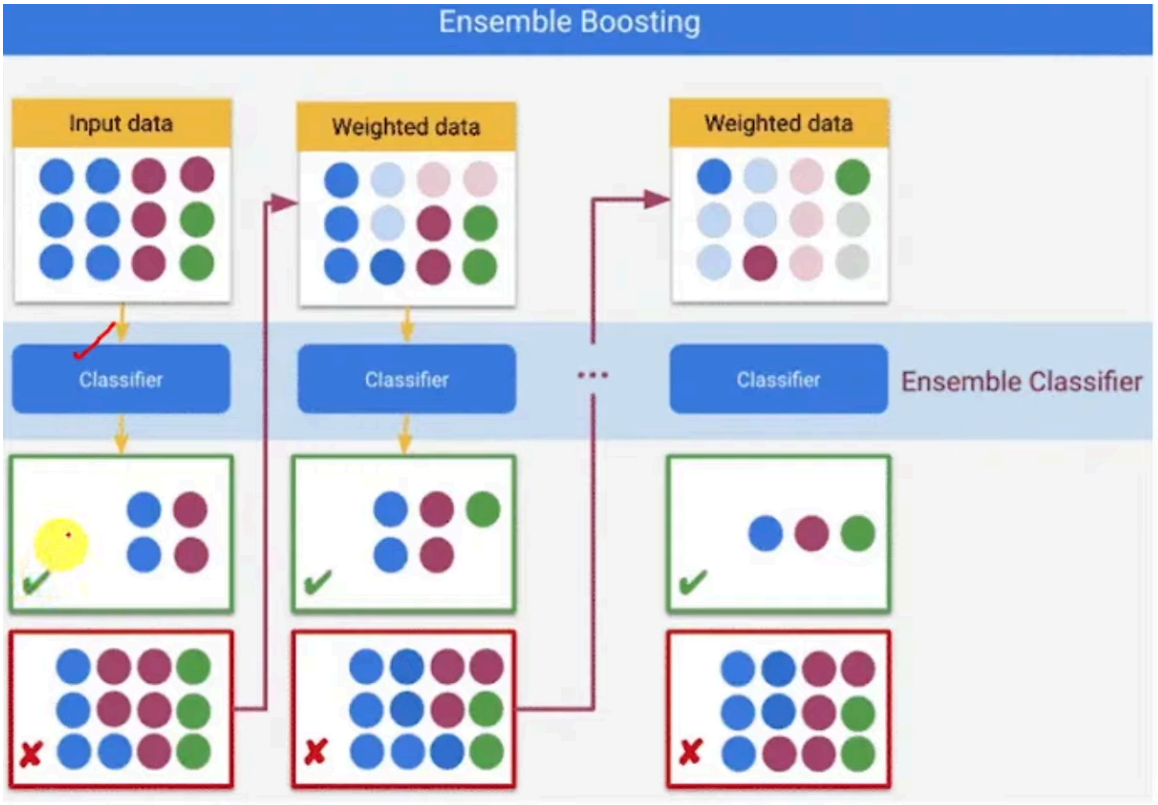
- **AdaBoost**: Boosts weak learners by reweighting misclassified instances, focusing more on the difficult cases.
- **XGBoost**: A more advanced and optimized version of boosting that improves on AdaBoost by controlling overfitting through regularization.

Advantages of Boosting:

- **Reduces Both Bias and Variance**: Boosting tends to improve both bias and variance of the model, making it effective for high-bias models that need to be refined.
- **Highly Accurate**: Boosting is often more accurate than bagging because it builds the models sequentially, allowing it to focus on the hardest-to-predict cases.
- **Focuses on Harder Instances**: Boosting emphasizes difficult-to-predict instances by re-weighting them, which can help achieve better performance, especially in complex datasets.
- **Works Well with Weak Learners**: Boosting can turn a weak learner (a model that performs just slightly better than random guessing) into a strong learner.

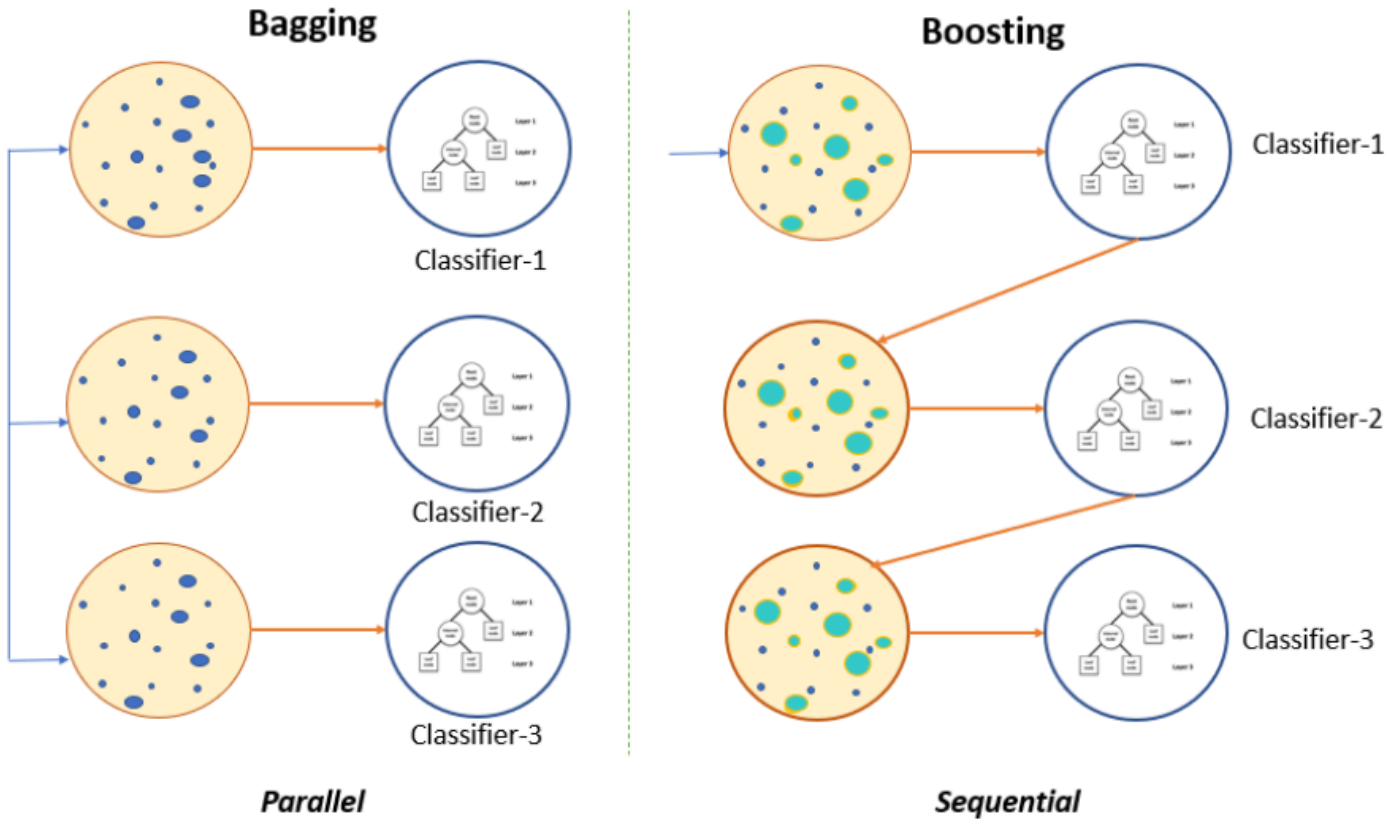
Disadvantages of Boosting:

- **Prone to Overfitting**: Boosting is sensitive to noisy data and outliers. Since it adjusts weights to focus more on difficult instances, it might overfit the noise in the dataset.
- **Computationally Expensive**: Boosting is sequential, meaning each model depends on the previous one. This leads to longer training times, especially with large datasets.
- **Requires Careful Tuning**: Boosting models (e.g., XGBoost) often require careful tuning of parameters like learning rate, the number of estimators, and tree depth to prevent overfitting and achieve optimal performance.
- **Less Parallelizable**: Unlike bagging, boosting is harder to parallelize because models are built sequentially. This makes it slower to train when compared to bagging, especially on large datasets.



Comparison of Bagging vs Boosting

Feature	Bagging	Boosting
Training Method	Parallel (independent models)	Sequential (models depend on each other)
Focus	Reduces variance by averaging predictions	Reduces both bias and variance, focusing on misclassified instances
Base Learner	Typically uses simple models (e.g., decision trees)	Typically uses weak learners, often decision trees
Example Algorithms	Random Forest, Bagged Decision Trees	AdaBoost, XGBoost, Gradient Boosting
Parallelization	High (models are independent)	Low (models are dependent)
Risk of Overfitting	Lower, but depends on the base model	Higher, especially if parameters aren't tuned well
Performance	Good at reducing variance, but not bias	Typically provides better accuracy by reducing both bias and variance
Computational Cost	Less expensive (due to parallelization)	More expensive (due to sequential training)



Random Forest is a powerful, versatile supervised machine learning algorithm based on ensemble learning. It improves prediction accuracy by combining the results of multiple decision trees.

Key Features:

1. **Ensemble Learning**:
   - Combines multiple decision trees to create a "forest."
   - Uses **bagging** (Bootstrap Aggregating) to reduce variance and improve accuracy.
2. **Versatility**:
   - Works for both **classification** (categorical output) and **regression** (continuous output) tasks.
3. **Randomness**:
   - Two levels of randomness are introduced:
     - **Random sampling** of data for each tree.
     - **Random subset of features** for each split in the tree.

How Random Forest Works:

1. **Pick N random records** from the dataset (with replacement).
   - This creates a unique training set for each tree (bootstrapped dataset).
2. **Build a decision tree** using the randomly selected records.
   - At each split, consider a random subset of features instead of all features.
3. **Repeat steps 1 and 2** for the desired number of trees.
   - Typically, dozens or hundreds of trees are grown to form a "forest."
4. **Make predictions**:
   - **For regression**: Each tree predicts a value, and the final prediction is the **average** of all tree predictions.
   - **For classification**: Each tree votes for a class, and the final prediction is the class with the **majority vote**.

Advantages of Random Forest:

1. **Reduces Overfitting**:
   - By averaging predictions, it reduces the variance compared to individual decision trees.
2. **Handles Missing Data**:
   - Can maintain accuracy even with missing values or noisy data.
3. **Feature Importance**:
   - Provides insight into which features are most important for making predictions.
4. **Versatile**:
   - Works for both classification and regression and handles large datasets well.

Disadvantages of Random Forest:

1. **Computational Complexity**:
    - Building many trees can be slow and require more memory, especially for large datasets.
2. **Interpretability**:
    - Less interpretable than a single decision tree; the results are more like a "black box."
3. **Not Ideal for High-Dimensional Sparse Data**:
    - Works better with datasets where features are more numerous but not sparse.

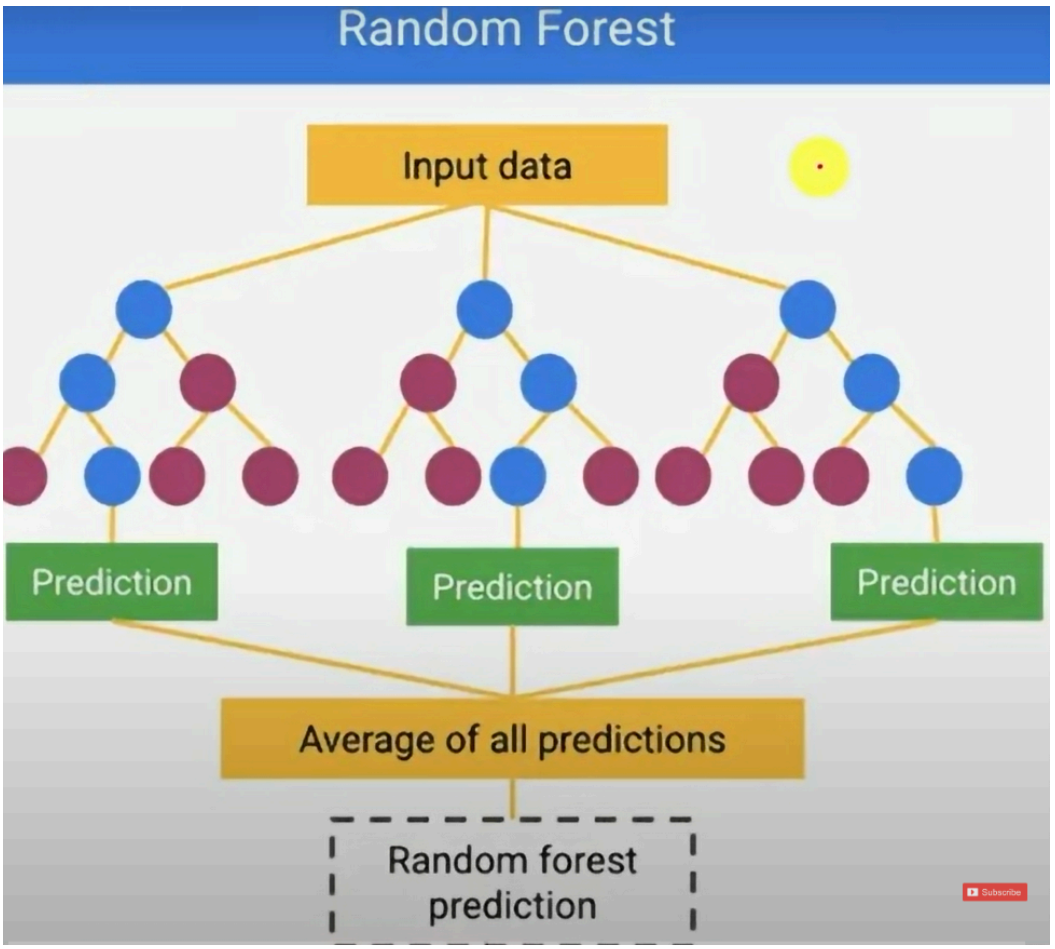
Applications of Random Forest:

- **Classification**: Spam detection, medical diagnosis, sentiment analysis.
- **Regression**: Predicting stock prices, real estate values, and sales forecasting.

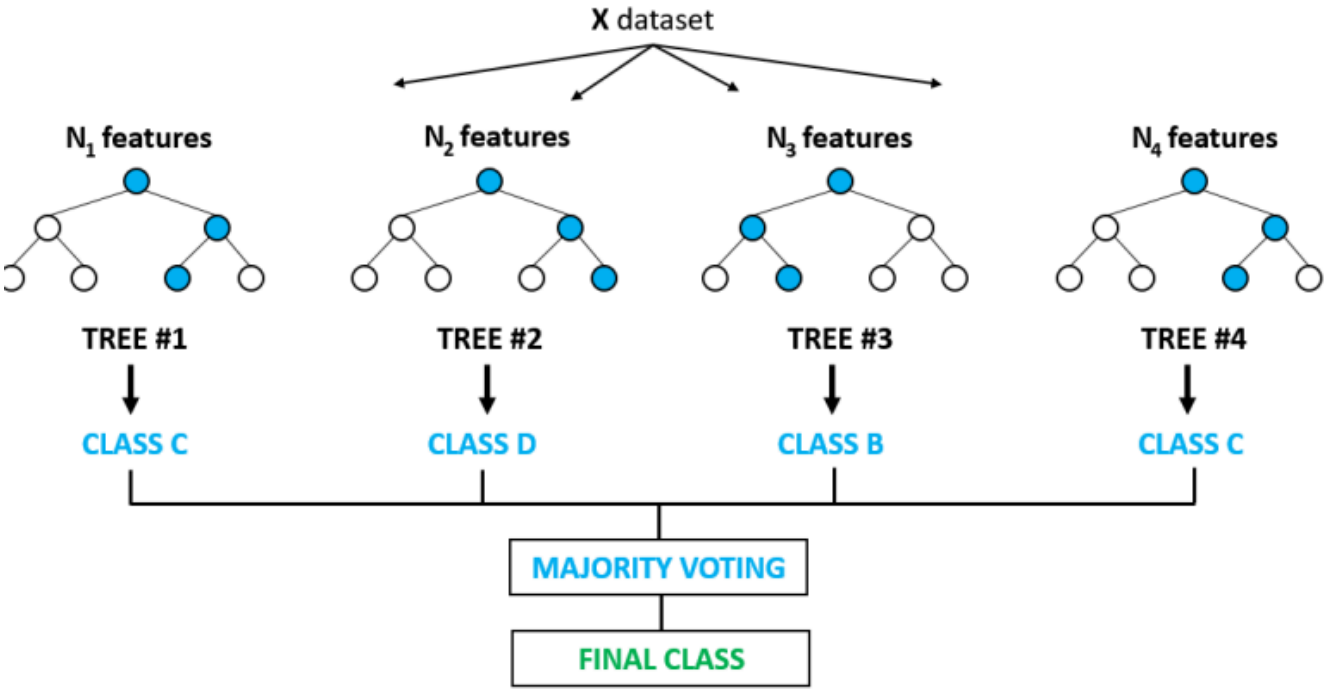
Out-of-bag (OOB) evaluation in random forests :

1. **No Extra Data Splitting**: It automatically uses leftover data (not used to build each tree) to test the model, so you don't need a separate validation set.
2. **Better Data Usage**: The entire dataset is used efficiently, which is great for small datasets.
3. **Unbiased Results**: OOB error is calculated on data not seen during training, giving a fair measure of model performance.
4. **Saves Time**: It's built into the training process, so you don't need to run separate validation steps. Built into training process
5. **Guides Tuning**: OOB error helps adjust the model settings (like the number of trees) for better results.

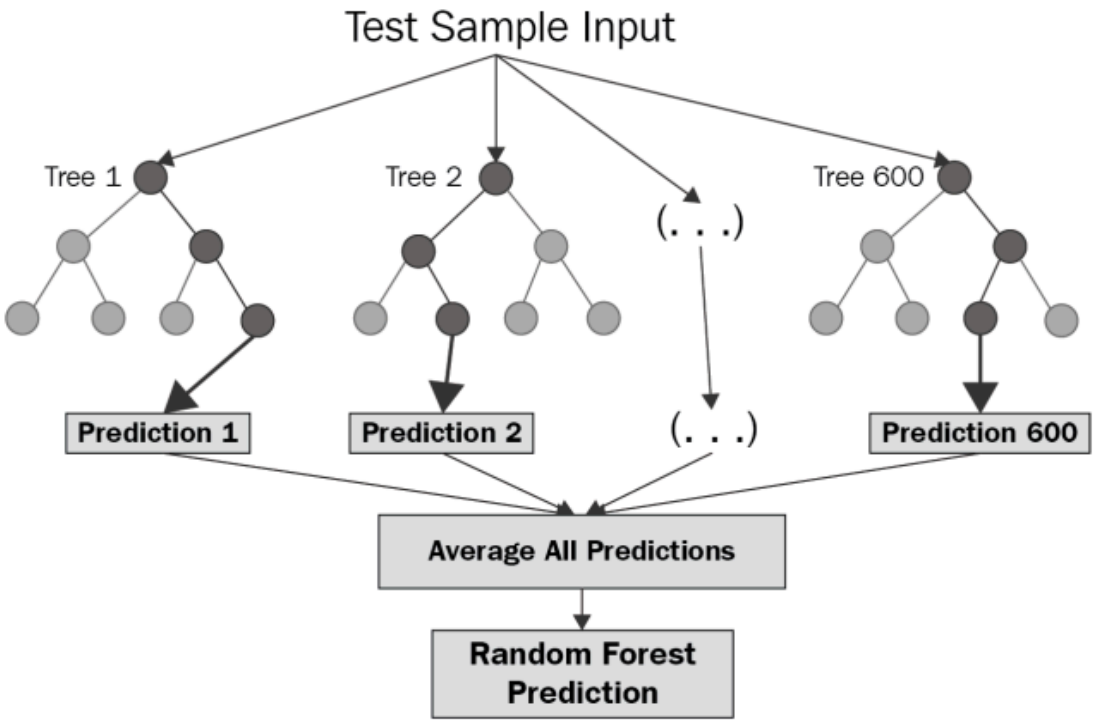
It's an easy and reliable way to check how well the random forest model works



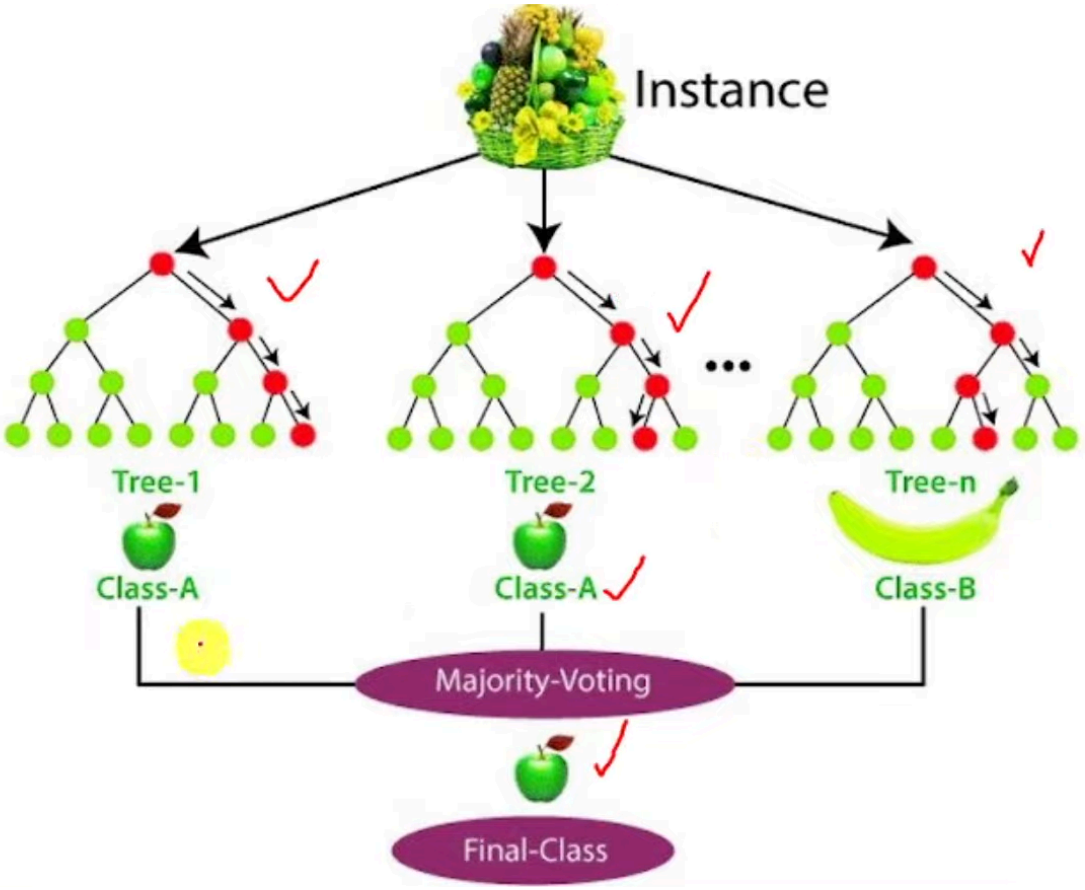
Majority Voting(classification output)



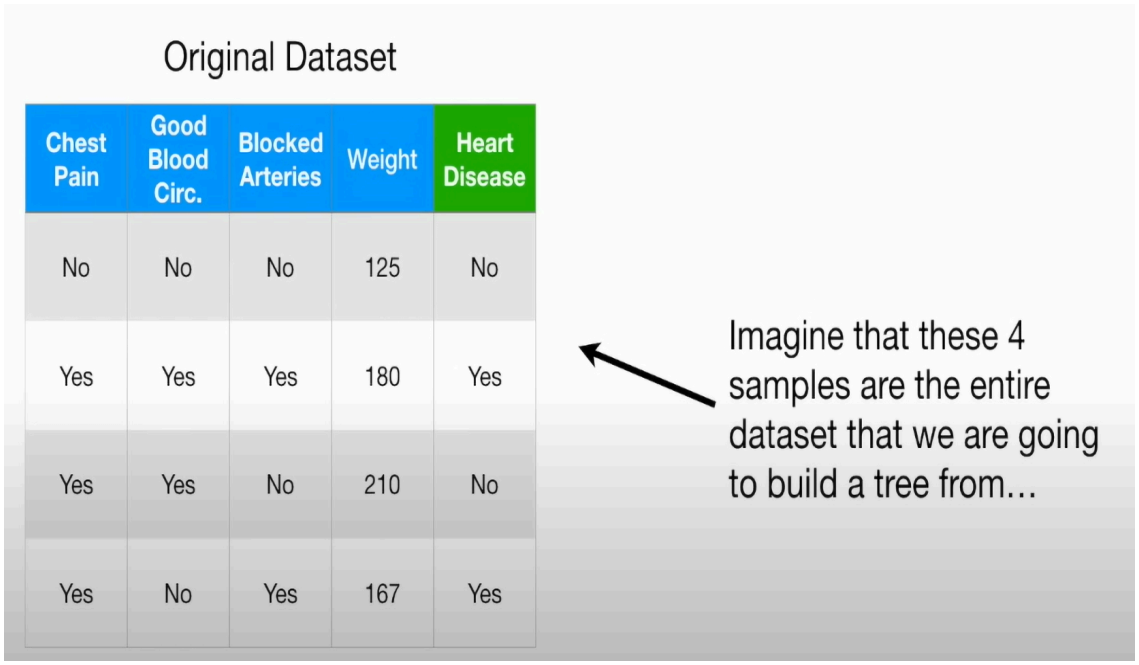
Regressor output:

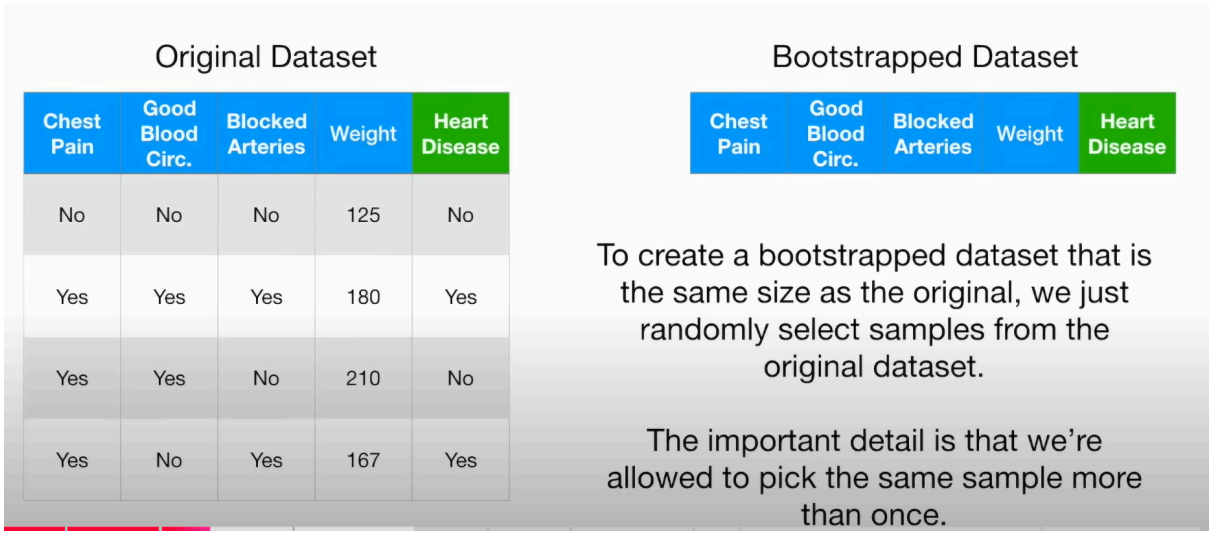


Example of Random forest:



Example 2:https://www.youtube.com/watch?v=J4Wdy0Wc_xQ







Step 2: Create a decision tree using the bootstrapped dataset, but only use a random subset of variables (or columns) at each step.

In this example, we will only consider 2 variables (columns) at each step.

NOTE: We'll talk more about how to determine the optimal number of variables to consider later…

Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

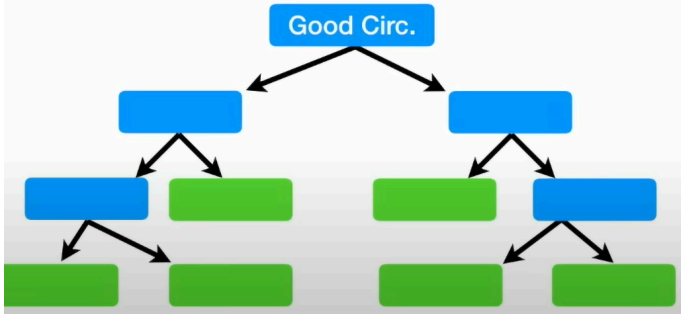
We built a tree…

1) Using a bootstrapped dataset

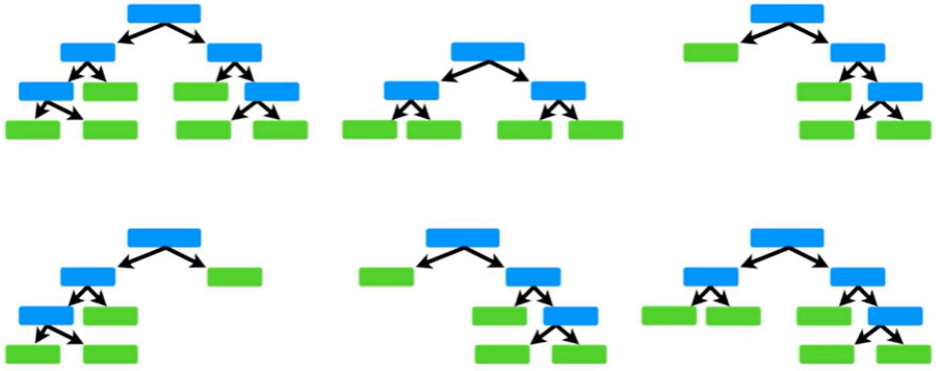
2) Only considering a random a subset of variables at each step.

Bootstrapped Dataset

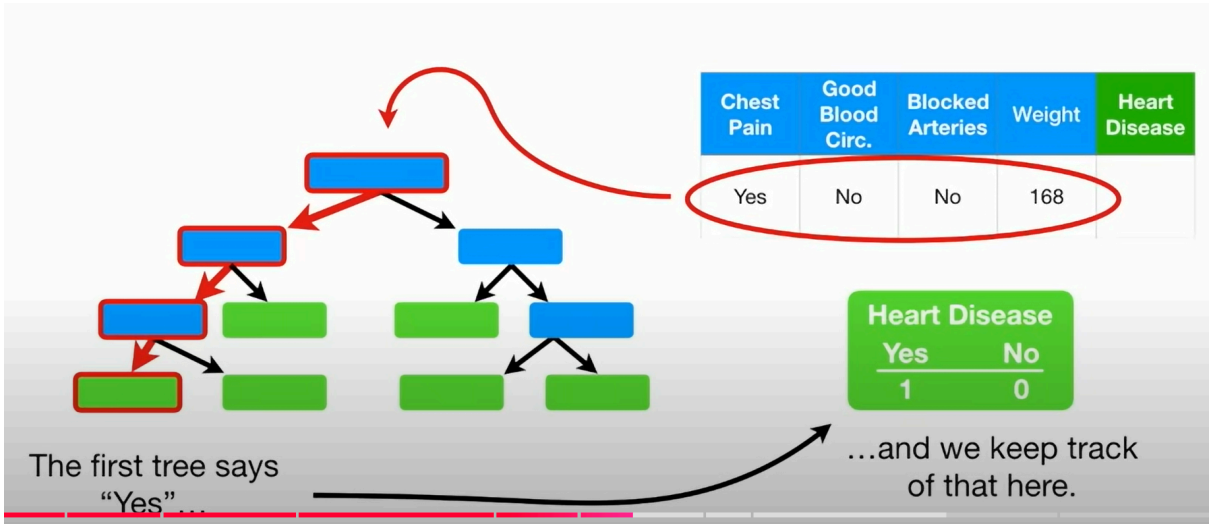
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

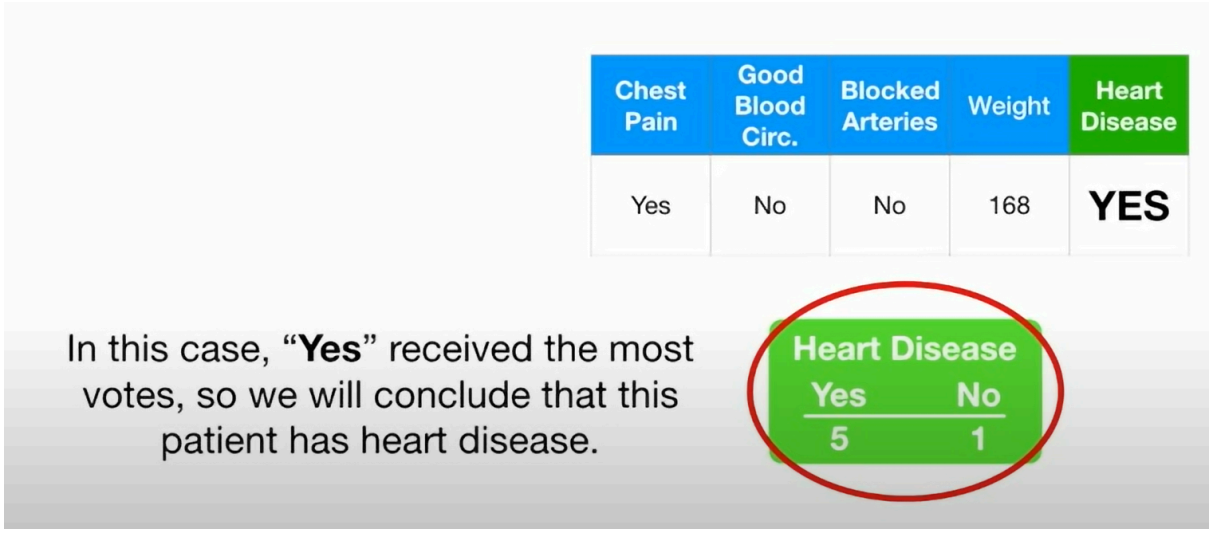


Now go back to Step 1 and repeat: Make a new bootstrapped dataset and build a tree considering a subset of variables at each step.



prediction





AdaBoost (Adaptive Boosting) Overview

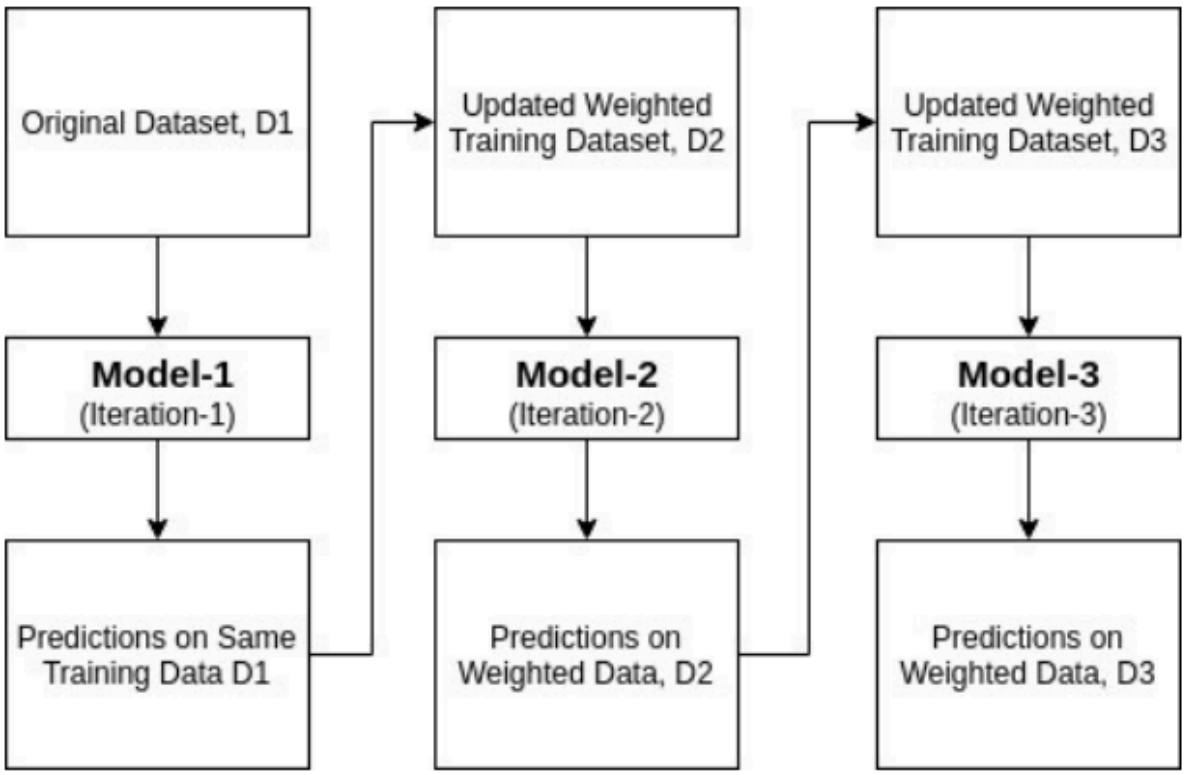
AdaBoost is an ensemble learning algorithm that focuses on **boosting** the performance of weak learners (usually simple models like decision stumps) by iteratively correcting errors.

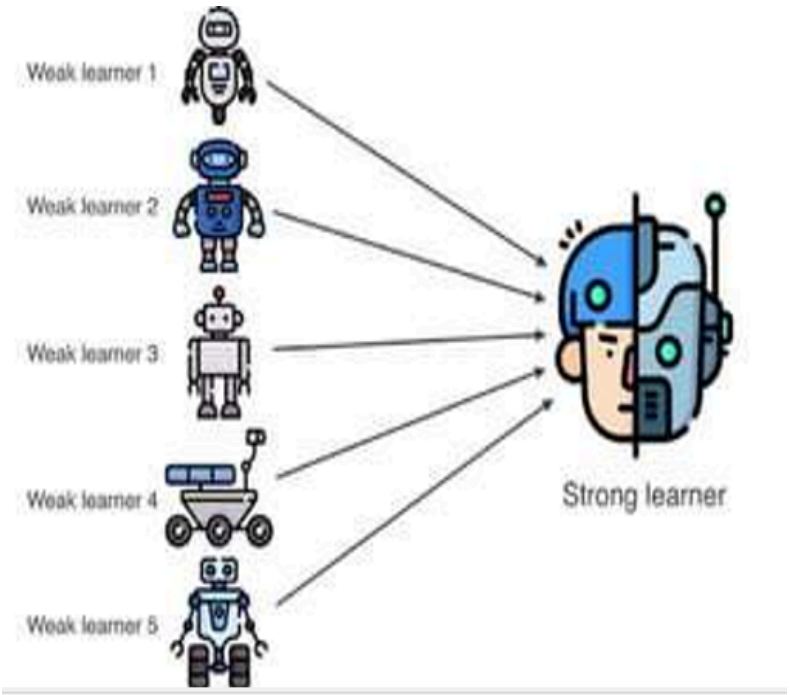
Key Features:

1. **Focus on Errors**:
   - Misclassified examples are given higher weights in subsequent iterations.
2. **Weighted Voting**:
   - Combines the predictions of all weak learners using weighted voting, where more accurate models contribute more to the final prediction.
3. **Iterative Training**:
   - AdaBoost iteratively updates the weights of the classifiers and data points to minimize errors.

How AdaBoost Works:

1. **Initialize Weights**:
   - Assign equal weights to all training samples initially.
2. **Train Weak Learner**:
   - Train a weak classifier (e.g., a decision stump) on the training data.
3. **Evaluate Performance**:
   - Calculate the error rate of the weak learner.
   - Higher weight is given to misclassified samples, making them more likely to be selected in the next iteration.
4. **Adjust Classifier Weights**:
   - Assign a weight to the weak learner based on its accuracy. More accurate classifiers get higher weights.
5. **Update Data Weights**:
   - Increase the weights of misclassified examples, so the next weak learner focuses on these harder cases.
6. **Repeat Steps 2–5**:
   - Continue until either:
     - All data points are correctly classified, or
     - The maximum number of weak learners is reached.
7. **Final Prediction**:
   - For classification, AdaBoost uses a weighted majority vote across all weak classifiers to make the final prediction.





Advantages of AdaBoost:

1. **Improves Accuracy**:
   - Boosts the performance of weak classifiers into a strong ensemble.
2. **Versatile**:
   - Can be used with various weak learners, not limited to decision stumps.
3. **Focus on Difficult Samples**:
   - Prioritizes harder-to-classify examples, improving robustness.
4. **No Overfitting in Many Cases**:
   - Often resistant to overfitting, especially with smaller datasets.

Disadvantages of AdaBoost:

1. **Sensitive to Noisy Data and Outliers**:
   - Since misclassified points get higher weights, outliers can heavily influence the model.
2. **Computationally Intensive**:
   - Iterative nature increases training time, especially with large datasets.
3. **Requires Careful Parameter Tuning**:
   - Needs careful tuning of the number of estimators and learning rate.

Applications of AdaBoost:

- **Face Detection**: Widely used in object detection tasks like facial recognition.
- **Text Classification**: Used in spam detection and sentiment analysis.
- **Medical Diagnosis**: Helps in predicting diseases based on symptoms.

Gradient Boosting: is an ensemble technique that builds models sequentially, where each model corrects the errors of the previous one using gradient descent.

The name "gradient" in **gradient boosting** comes from using gradients (the direction of change) to minimize errors. Each new model is trained to reduce the errors from the previous one by following the gradient of the loss function, helping to improve predictions step by step.

How It Works:

1. **Start with a simple model** (e.g., mean value for regression).
2. **Calculate residuals** (errors) from the current model.
3. **Fit a weak learner** (small decision tree) to predict the residuals.
4. **Update the model** by adding the new learner's predictions, weighted by a learning rate (α).
5. **Repeat** until the error is minimized or the set number of iterations is reached.

Three steps to Gradient boosting :

1. **Optimize a Loss Function**
    - Gradient boosting focuses on minimizing a specific loss function (e.g., mean squared error for regression or log loss for classification). The loss function measures how well the model is performing and guides the learning process.
2. **Use Weak Learners (Decision Trees)**
    - Simple models, typically shallow decision trees, are used as weak learners. These trees are good at capturing patterns but individually have limited predictive power.
3. **Add Trees Sequentially**
    - Trees are added one at a time, and each tree corrects the errors (residuals) of the previous ones. This sequential addition gradually improves the model's accuracy.

Advantages:

- High accuracy.
- Flexible with different loss functions.
- Provides feature importance.

Disadvantages:

- Sensitive to overfitting.
- Computationally expensive.
- Requires careful tuning of hyperparameters.

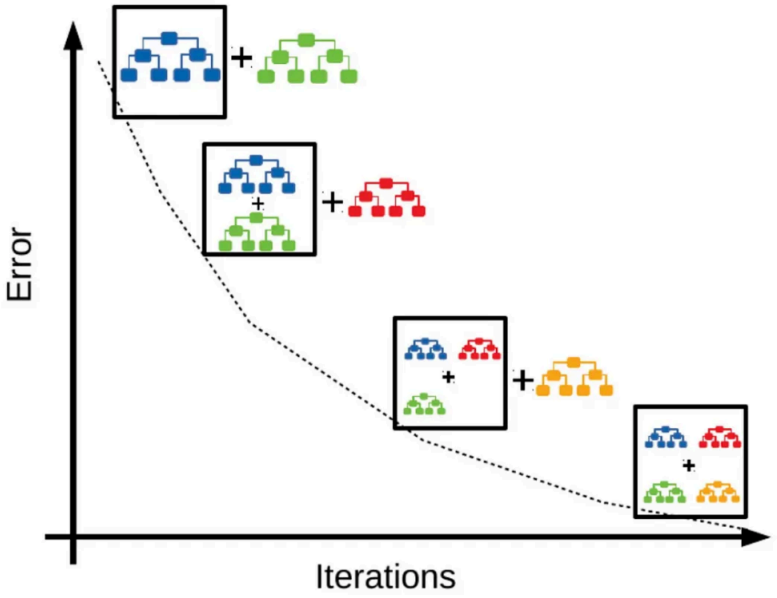
Applications:

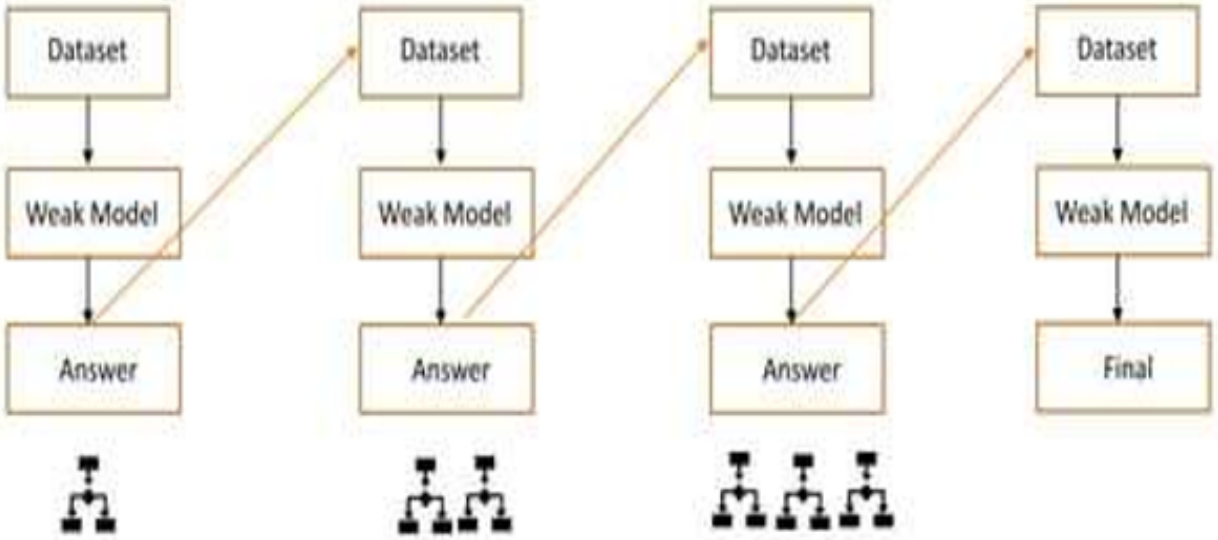
- **Finance**: Fraud detection, credit scoring.
- **Healthcare**: Disease prediction.

- **Marketing**: Customer churn analysis.

Popular Variants:

- **XGBoost**: Faster with regularization.
- **LightGBM**: Optimized for large datasets.
- **CatBoost**: Handles categorical data efficiently.





XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized implementation of Gradient Boosting designed for speed and performance. It improves on traditional Gradient Boosting by incorporating advanced features like regularization, parallel processing, and optimized memory usage.

Key Features of XGBoost:

1. **Speed and Efficiency**:
   - Uses parallel processing for faster training.
   - Optimized for both memory and computation.
2. **Regularization**:
   - Includes L1 (Lasso) and L2 (Ridge) regularization to prevent overfitting.
3. **Handling Missing Values**:
   - Automatically learns the best direction for missing values in splits.
4. **Sparsity Awareness**:
   - Efficiently handles sparse data (datasets with missing or zero values).
5. **Customizable Loss Functions**:
   - Supports user-defined objective functions for flexibility.

How XGBoost Works:

1. **Initialize** with a base model (e.g., mean for regression).
2. **Calculate residuals** (errors).
3. **Fit a new tree** to the residuals.
4. **Update model predictions** using the new tree, adjusted by a learning rate.
5. **Repeat** until a set number of iterations or stopping criteria is reached.

Key Parameters in XGBoost:

1. **Learning Rate (η)**:
   - Controls the step size for each update. Lower values improve generalization but require more iterations.
2. **Max Depth**:
   - Limits the depth of each tree to control overfitting.
3. **n_estimators**:
   - Number of trees in the ensemble.
4. **Regularization Parameters**:
   - **λ** (L2 regularization) and **α** (L1 regularization) to prevent overfitting.
5. **Subsample**:
   - Proportion of data used for each tree, reducing variance and speeding up training.

Advantages of XGBoost:

1. **High Accuracy**:
   - Often achieves superior performance in machine learning competitions.

2. **Fast Training**:
    - Efficient use of hardware resources, leveraging parallelism.
3. **Prevents Overfitting**:
    - Regularization helps generalize well on unseen data.
4. **Handles Missing Data**:
    - Automatically deals with missing values effectively.

Disadvantages of XGBoost:

1. **Computationally Intensive**:
    - Can be resource-heavy for very large datasets.
2. **Complex Tuning**:
    - Requires careful tuning of hyperparameters for optimal performance.
3. **Black Box**:
    - Less interpretable compared to simpler models like decision trees.

Applications of XGBoost:

- **Finance**: Fraud detection, credit scoring.
- **Healthcare**: Disease prediction.
- **Marketing**: Customer churn prediction, recommendation systems.
- **Competitions**: Frequently used in machine learning competitions (e.g., Kaggle).

Ensemble model needs:

1. **Multiple Models**: Combine several weak learners (simple models like decision trees).
2. **Diversity**: Ensure models are diverse to improve performance.
3. **Combination Method**: Use voting (classification) or averaging (regression) to combine predictions.
4. **Bias/Variance Reduction**: Address bias (e.g., boosting) or variance (e.g., bagging) by focusing on model errors.
5. **Learning from Errors**: Boosting methods focus on misclassified data points, while bagging uses random subsets.
6. **Scalability**: Some methods can be parallelized for faster training (e.g., Random Forest, XGBoost).
7. **Hyperparameter Tuning**: Proper tuning of parameters like number of models, learning rate, etc.
8. **Sufficient Data**: Large datasets are often needed for training diverse models

Metrics:

1. **Accuracy**: Proportion of correct predictions (for classification).
2. **Precision**: True positive predictions divided by total predicted positives.
3. **Recall**: True positive predictions divided by total actual positives.
4. **F1-Score**: Harmonic mean of precision and recall.
5. **Mean Squared Error (MSE)**: Measures average squared differences between predicted and actual values (for regression).
6. **Root Mean Squared Error (RMSE)**: Square root of MSE, interpretable in the same units as the data.

Error Correction:

- **Bias-Variance Trade-off**: Balancing bias (error from overly simplistic models) and variance (error from overly complex models).
- **Regularization**: Techniques like L1 (Lasso) and L2 (Ridge) regularization are used to prevent overfitting by penalizing model complexity.
- **Cross-Validation**: Dividing data into subsets to test and validate models, reducing the risk of overfitting.